```
Product.delete_all
# . . .
Product.create!(title: 'Design and Build Great Web APIs',
  description:
    %{<p>
      <em>Robust, Reliable, and Resilient</em>
      APIs are transforming the business world at an increasing pace. Gain
      the essential skills needed to quickly design, build, and deploy
      quality web APIs that are robust, reliable, and resilient. Go from
      initial design through prototyping and implementation to deployment of
      mission-critical APIs for your organization. Test, secure, and deploy
      your API with confidence and avoid the “release into production”
      panic. Tackle just about any API challenge with more than a dozen
      open-source utilities and common programming patterns you can apply
      right away.
      </p>},
  image_url: 'maapis.jpg',
  price: 24.95)
# . . .
```

(Note that this code uses `%{...}`. This is an alternative syntax for double-quoted string literals, convenient for use with long strings. Note also that because it uses the Rails `create!` method, it’ll raise an exception if records can’t be inserted because of validation errors.)

To populate your `products` table with test data, run the following command:

```
depot> bin/rails db:seed
```

Now let’s get the product listing tidied up. Normally this would require multiple steps: creating a CSS style sheet, linking that style sheet to your HTML, defining a set of style rules within the new style sheet, connecting these rules to the page by defining an HTML `class` attribute on the page, and changing the HTML to make styling the page easier.

Fortunately, we installed Tailwind CSS support, which is ideal for rapid development. Instead of spending time managing global CSS classes and trying to understand and debug the scope of your changes, you safely style your HTML page through an extensive set of predefined CSS utility classes.